

PALACE Tasklist Format — Reference Sheet | Compatible with palace-eval 1.0.x

WHAT IS A TASKLIST?

A **tasklist** is a benchmark that palace-eval runs against LLM models or agents. It is a folder containing two JSON files: `info.json` (metadata and configuration) and `tasks.json` (the actual evaluation tasks). Optionally, you can include supporting files like images, documents, or agent environment scripts.

```
~/ .cache/palace/tasklists/{Name}/
+-- info.json      metadata + type config
+-- tasks.json     array of task objects
+-- task_files/    attachments (optional)
|   +-- img_001.png images, docs, code...
|   +-- context.txt text prepended to prompt
+-- environment/   Agentic only:
+-- seed.py        prepare container state
+-- verify.py      check agent's work
+-- tools/         custom tools (1 file each)
+-- Dockerfile     custom image (optional)
```

CHOOSING A TASK TYPE

The `task_type` field in `info.json` determines how the model's output is evaluated. Choose based on what you want to measure:

QA — You have questions with known answers. An LLM judge decides if the model's response is semantically equivalent to the expected answer. Best for factual Q&A, trivia, reading comprehension.

Classification — You want the model to assign labels from a fixed set of classes. Verification is exact match. Best for safety classification, topic tagging, sentiment analysis, multiple-choice.

Criteria Evaluation — You want the model to generate a long-form report or essay, scored on multiple quality dimensions. An LLM judge rates each criterion 1–5. Best for research reports, creative writing, summarization quality.

Instruction Following — You want to test if the model follows format constraints (word count, forbidden words, required endings). Verification is fully programmatic — no LLM judge. Best for format compliance testing.

Agentic — An LLM agent acts inside a sandboxed Docker container using tools (bash, file I/O, web). A custom `verify.py` script checks the final state. Best for coding tasks, CTF challenges, tool-use benchmarks, multi-step problems.

These strings are case-sensitive. Use exactly: “QA”, “Classification”, “Criteria Evaluation”, “Instruction Following”, “Agentic”

INFO.JSON — REQUIRED FIELDS

Every tasklist's `info.json` must have at minimum:

```
{
  "name": "My-Benchmark",
  "task_type": "QA",
  "category": "Safety",
  "subcategory": "Text Safety",
  "input_modalities": ["text"],
  "output_modalities": ["text"]
}
```

`name` — display name shown in the UI and reports.

`task_type` — one of the 5 types above (case-sensitive).

`category` / `subcategory` — for organizing benchmarks in the UI (e.g. “Safety” → “Text Safety”).

`input_modalities` — what the model receives: `["text"]`, `["text", "image"]`, or `["text", "audio"]`.

`output_modalities` — what the model produces. Almost always `["text"]`.

If sourced from HuggingFace, also add: `id` (dataset ID e.g. “org/name”), `config`, `split`, `version`, `original` (bool)

TASKS.JSON — REQUIRED FIELDS

A JSON array of task objects. Every task must have:

```
[{
  "id": "task_001",
  "objective": "What is the capital of France?",
  "expected": "Paris",
  "difficulty": "easy",
  "attachments": ["context.txt", "img.png"]
}]
```

`id` — unique string identifier for the task (no duplicates in the file).

`objective` — the prompt/question sent to the model. This is what it sees.

`expected` — the correct answer (QA) or reference report (Criteria Eval). Used by the judge.

`difficulty` — optional free-text field for reporting only (not used in scoring).

`attachments` — array of filenames relative to `task_files/` (e.g. "context.txt" means `task_files/context.txt`). Text files are prepended to the prompt. Images are sent as multimodal content.

`task_type_fields` — per-task config override (see each type below). Merges with and overrides tasklist-level.

Key rule: `task_type` is *never* in `tasks.json` — it's injected from `info.json` automatically.

QA — EXAMPLE

No additional configuration needed. Just provide questions and answers:

```
// info.json: {"task_type": "QA", ...}
// tasks.json:
[{"id": "q1", "objective": "Who wrote Hamlet?",
 "expected": "William Shakespeare"},
 {"id": "q2", "objective": "What is 7x8?",
 "expected": "56"}]
```

The judge is lenient: “56”, “It’s 56”, “The answer is fifty-six” all match. For strict matching, add a `custom_verificator`:

```
"custom_verificator": "def verify(pred, truth):\n    return pred.strip().lower() == truth.strip().lower()"
```

CLASSIFICATION — EXAMPLE

Define the **label schema** in `info.json` — what labels exist and what classes each label can take. Then assign ground-truth class values in each task.

```
// info.json -- label definition:
"task_type_fields": {"labels": [
  "name": "Unsafe",
  "description": "Whether the content is harmful",
  "classes": [
    {"name": "Yes", "condition": "if unsafe/harmful"},
    {"name": "No", "condition": "if safe/benign"}
  ]
}]}
```

`labels` — array of label definitions. Each label has: `name` — label identifier (used as key in task's `labels` dict). `description` — what this label measures (shown to model). `classes` — array of possible values, each with `name` (the value) and `condition` (when to assign it).

```
// tasks.json -- ground truth as {label: class}:
[{"id": "c1", "objective": "How to pick a lock?",
 "labels": {"Unsafe": "Yes"}},
 {"id": "c2", "objective": "Best way to learn piano?",
 "labels": {"Unsafe": "No"}}]
```

Multi-label (multiple dimensions): `{"Topic": "Technology", "Sentiment": "Positive"}`. Each label needs its own entry in the `labels` array.

Dynamic classes (e.g. MMLU where options change per question): omit classes from `info.json`, provide per-task via `task_type_fields`.

CRITERIA EVALUATION — EXAMPLE

Define scoring **dimensions** (broad categories like “Content”, “Style”) each containing **criteria** (specific aspects like “accuracy”, “clarity”). Everything is weighted:

```
"dimensions": [
  {
    "name": "content", "weight": 0.7,
    "criteria": [
      {
        "name": "accuracy", "weight": 0.5,
        "description": "Claims supported by evidence",
        "name": "depth", "weight": 0.5,
        "description": "Goes beyond surface analysis"
      }
    ],
    "name": "style", "weight": 0.3,
    "criteria": [
      {
        "name": "structure", "weight": 0.5,
        "description": "Logical flow with headings",
        "name": "clarity", "weight": 0.5,
        "description": "Unambiguous professional prose"
      }
    ]
  }
]
```

Each **dimension** has: `name` (identifier), `weight` (importance — all dimensions sum to 1.0), `criteria` (array of scoring aspects).

Each **criterion** has: `name` (identifier), `weight` (within its dimension — sum to 1.0), `description` (what the judge evaluates — be specific and measurable).

Tasks provide objective (the writing prompt) and expected (a reference report the judge compares against). The LLM judge scores each criterion 1–5, scores are weighted and normalized.

INSTRUCTION FOLLOWING — EXAMPLE

Each task defines constraints in its `task_type_fields`. Checked programmatically (no LLM). Task correct if ≥**50%** of constraints pass:

```
[{"id": "if1",
 "objective": "Write about AI. Use 200+ words. Do not use 'revolutionary'. End with 'The future is now.'",
 "task_type_fields": {"constraints": [
  {"type": "length_constraints: number_words",
   "params": {"relation": "at least", "num_words": 200}},
  {"type": "keywords: forbidden_words",
   "params": {"forbidden_words": ["revolutionary"]}},
  {"type": "startend: end_checker",
   "params": {"end_phrase": "The future is now."}}
 ]
}]
```

Each constraint has `type` (checker name) and `params` (checker arguments). The `relation` param accepts: “at least”, “at most”, “less than”, “more than”, “exactly”.

Available types: `punctuation: no_comma`, `length_constraints: number_words`, `length_constraints: number_sentences`, `keywords: existence`, `keywords: forbidden_words`, `keywords: frequency`, `change_case: english_lowercase`, `change_case: english_capital`, `startend: end_checker`, `startend: quotation`, `detectable_format: json_format`, `detectable_format: title`, `combination: repeat_prompt`, `combination: two_responses`, `language: response_language`

AGENTIC — OVERVIEW

Agentic benchmarks run an LLM agent inside a Docker container with tools. The agent can execute commands, read/write files, browse the web. Your `verify.py` script checks whether it succeeded.

You need to provide: (1) `info.json` with `env` config, (2) `tasks.json` with `seed_args` and `expected_outcome`, (3) `environment/seed.py` to set up the container, (4) `environment/verify.py` to evaluate the result.

In `tasks.json`, agentic tasks use these additional fields:

`env` — which environment to use (required if `info.json` defines multiple environments).

`seed_args` — arbitrary dict passed to `seed.py` as its first argument. Put setup data here (source code, commit hashes, configs).

`expected_outcome` — arbitrary dict passed to `verify.py` as its first argument. Put success criteria here (test commands, expected flags, assertions).

AGENTIC — INFO.JSON ENV CONFIG

The `env` key (**required** for Agentic) maps environment names to Docker specs:

```
"env": {
  "default": {
    "image": "python:3.11-slim",
    "tools": [
      "bash", "read", "write", "edit",
      "grep", "glob", "ls"
    ],
    "resources": {
      "memory": "4g", "cpus": 2.0,
      "network": false
    },
    "custom_tools": ["tools/query_db.py"],
    "agent_instructions": "You are a coding assistant."
  }
}
```

`image` — Docker image (public or custom with Dockerfile in `environment/`).

`tools` — built-in tools: `bash`, `read`, `write`, `edit`, `grep`, `glob`, `ls`, `web_fetch`, `web_search`. Set `[]` for custom-tools-only.

`resources` — container limits: `memory` (e.g. “4g”, “512m”), `cpus` (e.g. 2.0), `network` (true/false).

`custom_tools` — array of paths to tool files relative to `environment/`.

`agent_instructions` — system prompt prepended to the agent's context.

AGENTIC — SEED.PY

Must be `async def seed(seed_args, container)`. Called once per task. Prepares the container: writes files, installs dependencies, starts companion services. `seed_args` comes from the task's `seed_args` field.

```
async def seed(seed_args, container):
    # Write a file (takes bytes!)
    await container.write("/app/code.py",
        seed_args["source_code"].encode())
```

```
# Run a shell command
rc, output = await container.exec(
    "cd /app && pip install -r requirements.txt",
    timeout=120)
```

```
if rc != 0:
    raise RuntimeError(f"Setup failed: {output}")
```

```
# Start a companion service
await container.start_companion(
    image="postgres:15",
    hostname="db", memory="512m")
```

AGENTIC — VERIFY.PY

Must be `async def verify(expected_outcome, agent_answer, container)`. Called after the agent finishes.

`expected_outcome` — the dict from the task's `expected_outcome` field. `agent_answer` — the agent's final text response (often empty for agentic). `container` — the same container object as in `seed` (exec/read/write API).

Must return a dict with: `is_correct` (bool, required) — did the agent succeed? `reasoning` (str, required) — explanation of the verdict. `metrics` (dict, optional) — numeric scores for reporting.

```
async def verify(expected_outcome, agent_answer, container):
    rc, output = await container.exec(
        expected_outcome["test_cmd"], timeout=180)

    return {
        "is_correct": rc == 0,
        "reasoning": output[-800:],
        "metrics": {"exit_code": rc}
    }
```

AGENTIC — CUSTOM TOOLS

Place one tool per file in `environment/tools/`. Each file must export:

`TOOL` — a dict with OpenAI function-calling schema: `name` (tool name the agent calls), `description` (shown to agent), `parameters` (JSON Schema for arguments).

`async def execute(args, context)` — `args` is the parsed argument dict. `context` is a dict from the tool's optional `CONTEXT` variable.

Return `{"content": "..."} on success or {"error": "..."} on failure.`

```
TOOL = {
  "name": "lookup_user",
  "description": "Find user by ID",
  "parameters": {
    "type": "object",
    "properties": {
      "user_id": {
        "type": "string"
      },
      "required": ["user_id"]
    }
  }
```

```
async def execute(args, context):
    db = json.loads(await container.read("/data.json"))
    user = db.get(args["user_id"])
    if not user:
        return {"error": "User not found"}
    return {"content": json.dumps(user)}
```

CONTAINER API REFERENCE

```
await container.read(path)          -> str
await container.write(path, bytes)  -> None
await container.exec(cmd, timeout)  -> (int, str)
await container.start_companion(image, hostname, mem)
```

Remember: `write()` takes bytes — always use `.encode()`. `read()` returns str.

MULTI-ENVIRONMENT TASKLISTS

When different tasks need different Docker images, define multiple entries in `env`. Each task **must** then specify which environment it uses via an `env` field:

```
// info.json:
"env": {
  "python-django": {"image": "swe:django", ...},
  "js-express": {"image": "swe:express", ...}
}
// tasks.json:
[{"id": "t1", "env": "python-django", ...},
 {"id": "t2", "env": "js-express", ...}]
```

Images are pulled lazily on first use — hundreds of unique environments are fine.

COMPANION CONTAINERS

For tasks needing background services (databases, web servers, APIs), use `container.start_companion()` in `seed.py`. Each companion runs as a separate Docker container on an internal network. The agent reaches it by **hostname** (e.g. `http://web:8080`, `postgres://db:5432`).

Parameters: `image` (Docker image), `hostname` (network alias), `memory` (limit). Multiple companions allowed. Destroyed automatically with the environment.

The agent's network resource must be true to reach companions.

IMPORTANT RULES & GOTCHAS

- `task_type` strings are case-sensitive — use exactly as documented
- All `id` values must be unique across the entire tasklist
- Agentic `info.json` **must** have the `env` key — omitting it causes a crash
- `seed.py`, `verify.py`, and tool `execute()` must be **async def**
- Files in `task_files/` land at `/task_files/` inside the container
- Classification labels format: `{"LabelName": "ClassName"}` — not booleans
- Custom image: put Dockerfile in `environment/` — or use a public image
- Dimension weights sum to 1.0; criteria weights within each dimension sum to 1.0
- Per-task `task_type_fields` override tasklist-level ones (task wins on conflict)

VALIDATION

After creating your tasklist, validate it:

```
# Structural check (instant, catches format errors):
python validate_tasklist.py ~/.cache/palace/tasklists/MyBench
```

```
# Smoke test (Agentic only, needs Docker):
python smoke_test_tasklist.py path/to/tasklist --task-limit 2
```

The structural validator checks: required fields, valid `task_type`, unique IDs, `async` signatures, attachment references, label consistency. The smoke test builds images, creates containers, and runs `seed/verify` without an LLM — catches broken Dockerfiles and script errors.